

Sites LabMiM / LEAL

Onboarding da plataforma estática multi-publicação

Uma aplicação compartilhada para publicações meteorológicas de diferentes instituições, estados e produtos WRF.

LabMiM / UFBA · Bahia

LEAL / UFES · Espírito Santo

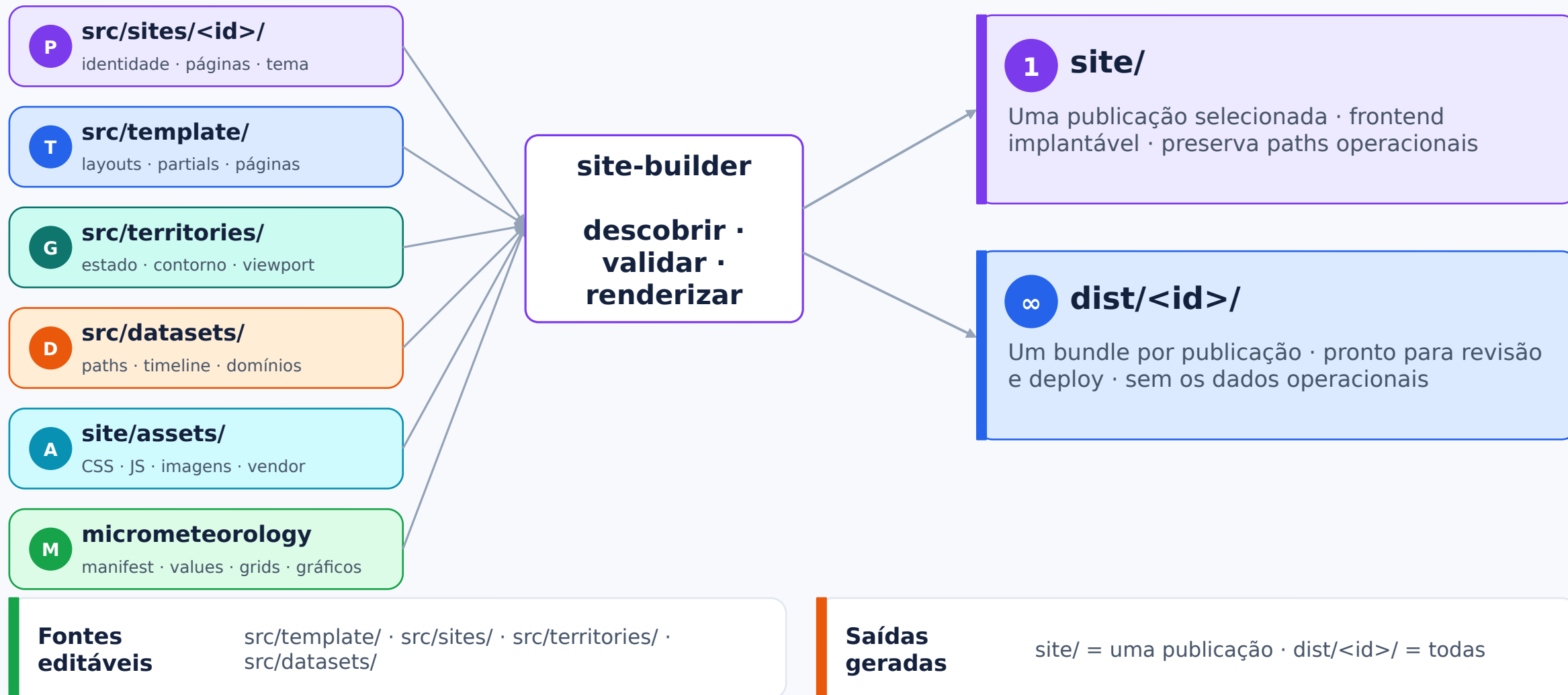
pronta para novas publicações

UMA BASE · VÁRIAS PUBLICAÇÕES

LabMiM
UFBA · BALEAL
UFES · ESNode 24 +
gerador próprioHTML + CSS
+ JS
purosLeaflet +
Chart.jsJSON
·
GeoJ
SON
serie
s.bin

Node só existe no build. O servidor continua inteiramente estático.

Mapa do repositório



O modelo mental: quatro módulos

P Publicação

Quem publica?

Conteúdo, SEO,
navegação e marca

```
src/sites/<id>/
```

T Template

O que é igual?

Layouts, partials,
páginas e estáticos
comuns

```
src/template/
```

G Território

Onde o mapa está?

Estado, contorno, centro,
zoom e fitBounds

```
src/territories/
```

D Dataset

De onde vêm os dados?

Paths, timeline, domínios
e atribuição WRF

```
src/datasets/
```

↻ Reutilização é esperada

Um território e um dataset podem ser usados por mais de uma publicação.

! Hardcode é um cheiro de arquitetura

Evite `if (site === "ufba")` no template, renderer, CSS e runtime.

Como o build compõe uma publicação



Descoberta: site.js é a composição

COMPOSIÇÃO DA PUBLICAÇÃO

```
// src/sites/ufes/site.js
"use strict";

const identity = require("../identity");
const pages = require("../pages");
const dataset = require("../../datasets/leal-wrf");
const territory = require("../../territories/es");

module.exports = { ...identity, territory, dataset, pages };
```

🔄 Registro automático

- uma pasta válida é o registro
- sites:list confirma a descoberta
- build:all e build:check usam a mesma lista

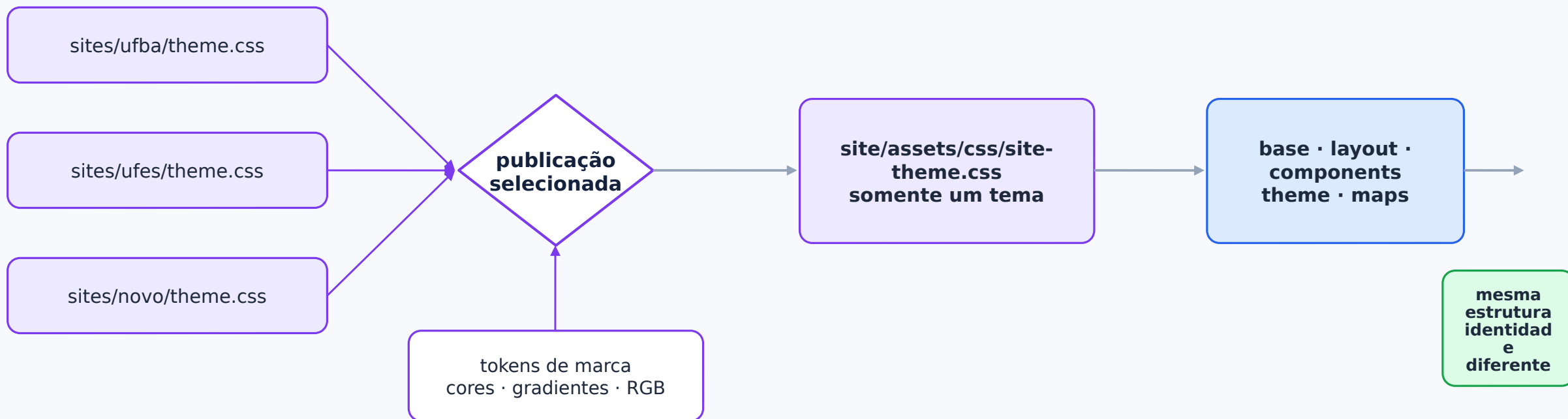
✓ Invariantes

- ID igual à pasta
- origin única, sem barra final
- exatamente um isDefault: true
- schemaVersion: 1

O filesystem é o registro

Adicionar uma publicação válida não exige editar build.js nem package.json.

CSS desacoplado por responsabilidade



Identidade

src/sites/<id>/theme.css
19 tokens, nenhum seletor.

Estrutura comum

base · layout · components · theme

Por página

vendorStyles + styles
WebGIS declara Leaflet e maps.css.

Onde colocar uma mudança de estilo?

SEMPRE CARREGADO

marca `src/sites/<id>/theme.css`

reset / utilitário `base.css`

navbar / footer `layout.css`

cards / blocos `components.css`

dark mode `theme.css`

DECLARADO POR PÁGINA

WebGIS `maps.css` em `styles`

asset comum `"assets/css/..."`

fonte compartilhada `templateSource("styles/...")`

fonte exclusiva `siteSource("styles/...")`

biblioteca local `vendorStyles`

Regra de ouro

Não duplique estrutura num tema e não adicione seletor por ID no CSS comum.

Comandos e saídas

COMANDOS

```
npm run sites:list
npm run build
npm run build -- --site=ufes
npm run build:all
npm run build:check
padrão
npm run lint:themes
```

padrão -> site/
escolhida -> site/
todas -> dist/<id>/
valida todas; restaura
contrato e isolamento

1 site/

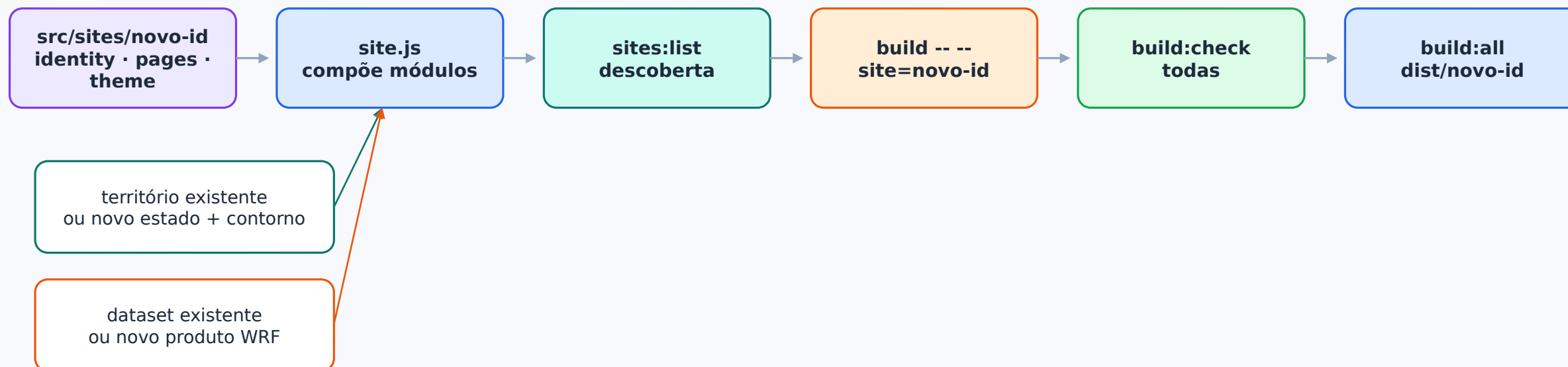
- uma publicação por vez
- compatível com o deploy atual
- preserva paths operacionais
- remove HTMLs órfãos

∞ dist/<id>/

- um frontend por publicação
- pronto para os dados corretos
- omite paths operacionais
- dist/ é git-ignored



Criar uma publicação nova: fluxo completo



Criar

```
src/sites/exemplo/  
├── site.js  
├── identity.js  
├── pages.js  
├── theme.css  
├── pages/  
│   ├── styles/ # opcional  
│   └── fragments/ # opcional
```

Não editar

- build.js
- package.json
- outputs derivados do manifesto
- template para trocar nomes de instituição
- CSS comum para trocar paleta

Passo 1: identidade

IDENTITY.JS

```
// src/sites/exemplo/identity.js
module.exports = {
  schemaVersion: 1,
  id: "exemplo",
  isDefault: false,
  origin: "https://exemplo.edu.br",
  brand: {
    name: "LAB",
    fullName: "Laboratório Exemplo",
    copyrightName: "LAB",
    ogImage: "assets/img/logo-exemplo.png",
    logos: { nav: { /* src + dimensões */ }, footer: { /* ... */ }, sidebar: { /* ... */ } },
    affiliations: [],
  },
  institution: { name: "Universidade Exemplo", acronym: "UE" },
  location: { cityName: "Cidade" },
  theme: "theme.css",
  redirects: [],
};
```

ID

Identificação

schemaVersion · id · isDefault · origin

B

Marca e instituição

logos · OG image · nome · sigla · afiliações

↗

Navegação e URL

theme · redirects · cidade e origem canônica

Regras

Assets apontam para site/assets/
redirects só chegam a páginas
declaradas.

Passo 2: território e dataset

src/territories/pe.js

```
module.exports = {
  id: "pe",
  kind: "state",
  code: "PE",
  name: "Pernambuco",
  regionPhrase: "região de Pernambuco e entorno",
  terrainExample: "o Planalto da Borborema",
  boundaryAsset: "assets/data/br_pe.json",
  viewport: {
    center: [-8.3, -37.8],
    zoom: 6,
    fitBoundary: true,
    fitMaxZoom: 7,
  },
};
```

src/datasets/exemplo-wrf.js

```
module.exports = {
  id: "exemplo-wrf",
  attribution: "LAB-UE",
  paths: {
    manifest: "JSON/manifest.json",
    values: "JSON",
    grids: "GeoJSON",
  },
  timeline: {
    defaultMaxLayer: 73,
    initialIndex: 7,
    stepHours: 1,
    label: "Horário local (UTC-03)",
  },
  defaultDomain: "D01",
  domains: [/* id, label, centro, zoom, resolução */],
};
```

Território = geografia estado · contorno · centro · zoom · textos geográficos

Dataset = contrato operacional

paths · timeline · domínios · atribuição

Montar o site

PAGES.JS

```
// src/sites/exemplo/pages.js
const { page, siteSource } = require("../../template/page-types");

module.exports = [
  page("home", {
    source: siteSource("pages/index.html"),
    seo: {
      hl: "LAB - Laboratório Exemplo",
      title: "LAB - Laboratório Exemplo · UE",
      description: "Pesquisa meteorológica da Universidade Exemplo.",
    },
  }),
  page("forecast", {
    seo: {
      title: "LAB - Previsões WRF · UE",
      description: "Previsões para Pernambuco.",
    },
  }),
];
```

19

Tema

Implemente os 19 tokens em theme.css.

js

Composição

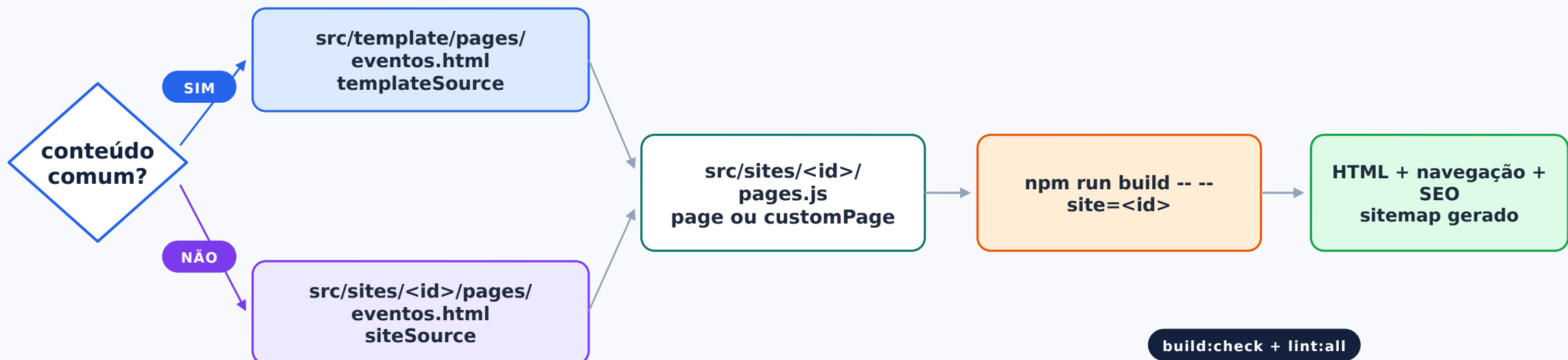
Junte identity, territory, dataset e pages no pequeno site.js.

✓

Build

sites:list → build individual → build:check.

Adicionar uma página: primeiro decida o alcance



Compartilhada

Fonte em `src/template/pages/`; cada publicação opta por incluí-la.

Exclusiva

Fonte em `src/sites/<id>/pages/`; só aquele manifesto referencia.

Receita: página compartilhada

MANIFESTO DA PÁGINA

```
// em src/sites/<id>/pages.js
const { customPage, templateSource } = require("../../template/page-types");

customPage({
  id: "about-project",
  file: "projeto.html",
  layout: "institutional",
  source: templateSource("pages/projeto.html"),
  seo: {
    h1: "Sobre o projeto",
    title: "Sobre o projeto · LAB",
    description: "Objetivos, metodologia e resultados.",
  },
  nav: {
    label: "Projeto", icon: "fa-circle-info", order: 60,
    elementId: "nav-projeto",
  },
  styles: [templateSource("styles/projeto.css")],
});
```

1 Crie a fonte

src/template/pages/projeto.html
Sem <html>, <head>, navbar ou footer.

2 Declare o uso

Cada publicação inclui a página em pages.js.

✓ Gere e valide

build da publicação + build:check + lint:all.

Receita: página exclusiva ou variação editorial

PÁGINA EXCLUSIVA

```
const { customPage, siteSource } = require("../..//template/page-types");

customPage({
  id: "projeto-local",
  file: "projeto-local.html",
  layout: "institutional",
  source: siteSource("pages/projeto-local.html"),
  styles: [siteSource("styles/projeto-local.css")],
  seo: {
    h1: "Projeto local",
    title: "Projeto local · LEAL/UFES",
    description: "Iniciativa exclusiva no Espírito Santo.",
  },
  nav: false,
});
```

+ Anexar um trecho próprio

```
append:
[siteSource("fragments/funding.html")]
```

↔ Personalizar um tipo comum

Sobrescreva source, nav, styles ou campos de SEO.

Escolha consciente Use siteSource somente quando o conteúdo ou estrutura realmente pertencer à publicação.

Atualizar uma página existente: mapa de impacto

CONTEÚDO E CATÁLOGO

comum src/template/pages/*.html

exclusivo src/sites/<id>/pages/*.html

anexo src/sites/<id>/fragments/*.html

SEO / H1 src/sites/<id>/pages.js

menu / ordem page.nav em pages.js

ESTRUTURA E IDENTIDADE

head / navbar / footer src/template/partials/

shell de página src/template/layouts/

CSS compartilhado site/assets/css/

CSS autoral templateSource / siteSource

paleta src/sites/<id>/theme.css



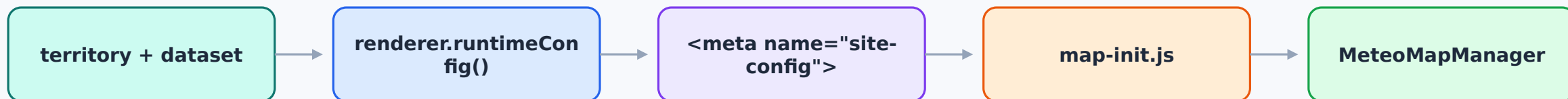
Antes de editar algo compartilhado

Procure todos os consumidores com rg e valide todas as publicações.

pages.js: uma fonte de verdade por site



Território e dataset chegam ao runtime



G Território controla

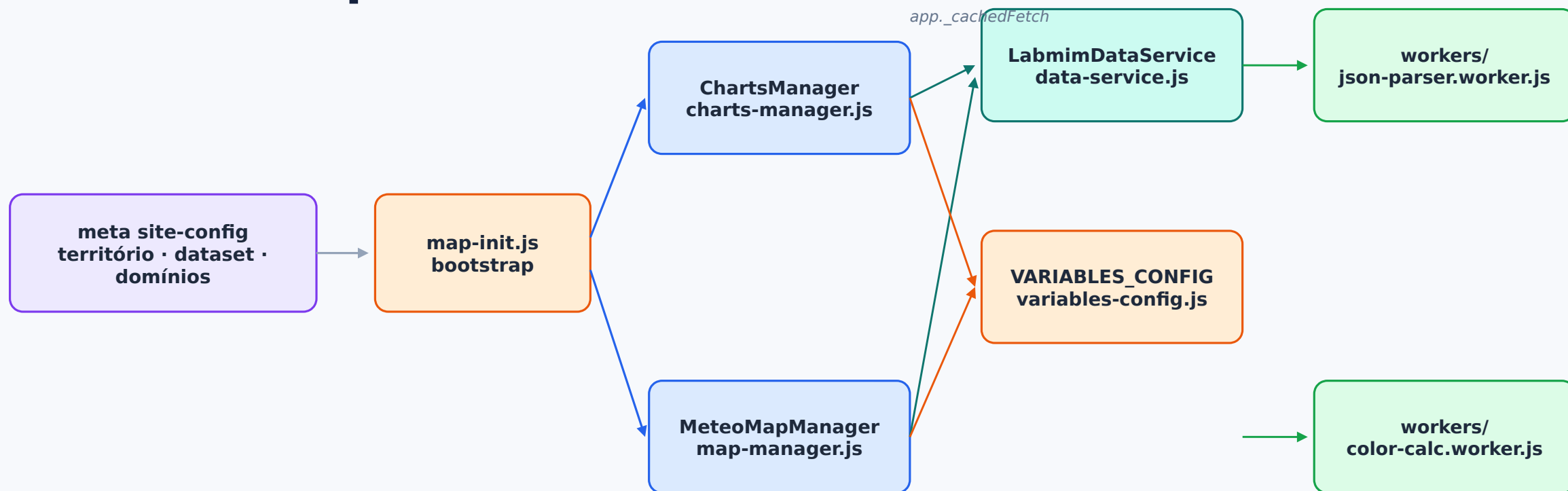
- estado e sigla
- contorno local
- centro e zoom inicial
- fitBounds e zoom máximo
- textos geográficos do template

D Dataset controla

- manifest, values e grids
- timeline fallback
- domínio padrão
- labels, centros e zooms
- documentação e atribuição

Resultado O runtime é genérico: a publicação injeta apenas configuração serializada.

WebGIS compartilhado



Reutilização forecast e energy usam o mesmo layout WebGIS; contexto, conteúdo, estilos e SEO vêm da declaração da página.

Dados e gráficos: o build não os produz

PNG

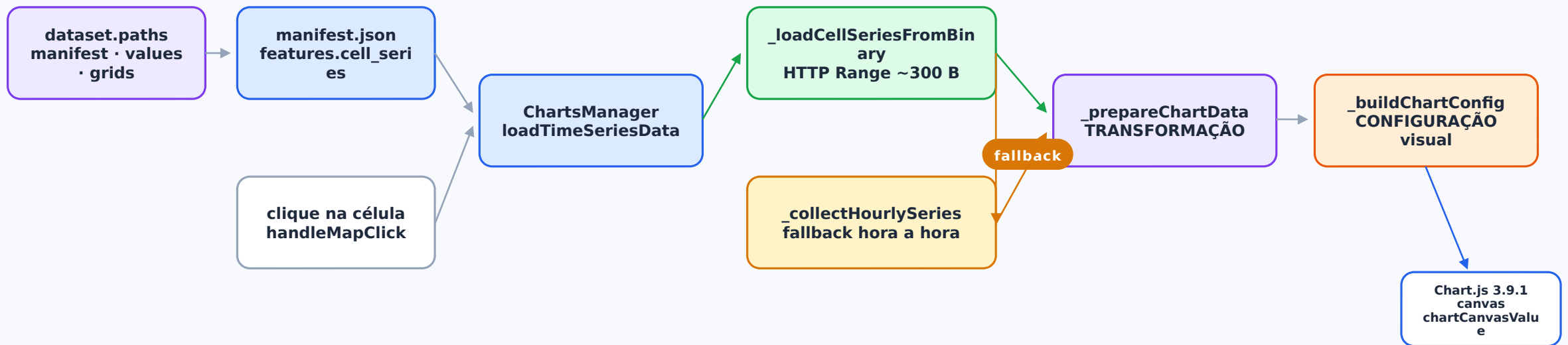
Monitoring

- 9 PNGs de nome fixo
- gerados por labmim-site-graphs
- cards e modais Bootstrap
- nenhuma instância Chart.js

GIS

WebGIS

- manifest + JSON/GeoJSON
- series.bin via HTTP Range
- summary.json para prévia
- Chart.js controlado por ChartsManager



O que build:check protege

R Registro

ID/pasta · origem · padrão

I Identidade

logos · URLs · redirects

T Tema

tokens · seletores · pares
hex/RGB

G Território

GeoJSON · sigla · viewport

D Dataset

paths · timeline · domínios

P Páginas

fonte · layout · style · vendor
· SEO/nav

O Saída

tokens · HTML · referências
locais

C CSS / vendor

Bootstrap purgado · ícones

Complementar

npm run lint:all · npm run format:check · inspeção manual em light/dark, mobile e links.

Riscos de regressão



Editar site/*.html

mudança perdida



Copiar página comum

conteúdo diverge



Esquecer manifesto consumidor

catálogo desigual



Testar CSS comum em um site

regressão temática



Identidade no template/CSS

marca vaza



siteSource para capacidade comum

duplicação



CSS sem page.styles

estilo ausente



Hardcode de estado/path

mapa ou dados errados



Prevenção

Fronteiras declarativas + build:check + inspeção de todas as publicações afetadas.

Checklists operacionais

NOVA PUBLICAÇÃO

- ✓ identity.js, pages.js, theme.css
- ✓ páginas/fragments exclusivos
- ✓ território + contorno, se novo
- ✓ dataset, se novo
- ✓ composição em site.js
- ✓ sites:list e build individual
- ✓ build:check e build:all
- ✓ dados operacionais corretos no deploy

PÁGINA OU ATUALIZAÇÃO

- ✓ decidir comum versus exclusiva
- ✓ editar fonte, não output
- ✓ atualizar SEO/nav em pages.js
- ✓ declarar styles, se houver
- ✓ verificar consumidores compartilhados
- ✓ build da publicação afetada
- ✓ build:check + lint:all
- ✓ light/dark, mobile e links

De onde vêm os dados: o repositório irmão

O site-labmim monta o frontend. O pacote micrometeorology lê WRF e estação, produz os artefatos e mantém o contrato de integração.



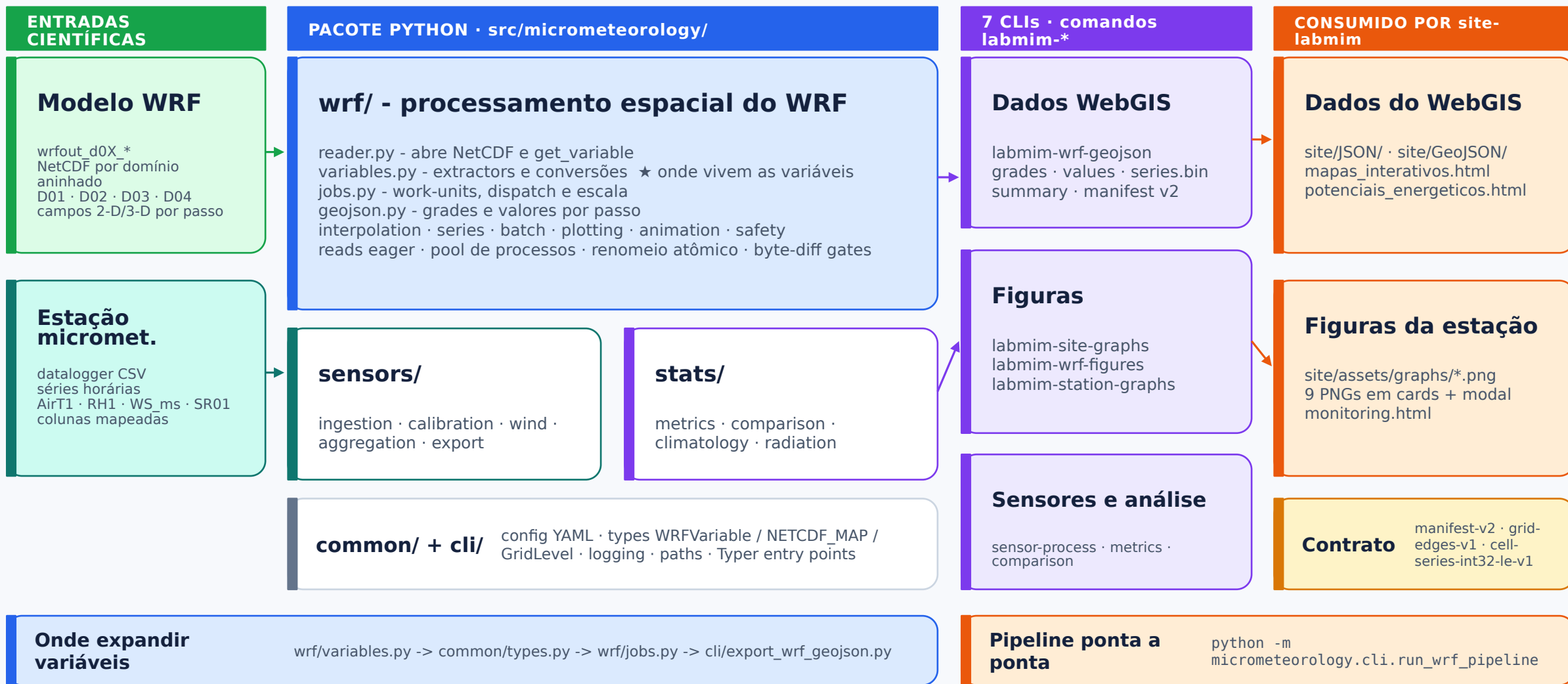
7 CLIs labmim-*

WebGIS: labmim-wrf-geojson
Figuras: labmim-site-graphs · labmim-wrf-figures · labmim-station-graphs
Sensores/análise: labmim-sensor-process · labmim-metrics · labmim-comparison

Só labmim-wrf-geojson alimenta diretamente os mapas interativos.

Pacote micrometeorology: do modelo ao artefato publicado

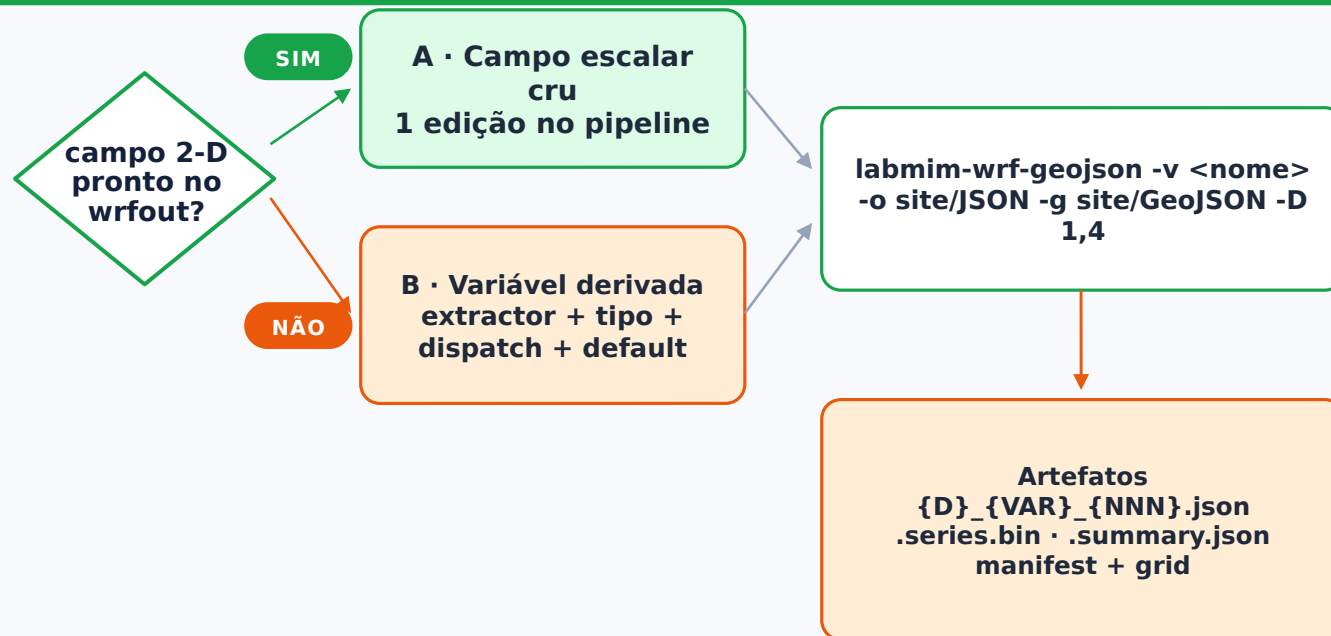
As camadas do pacote Python e as CLIs que produzem os dados e figuras do site



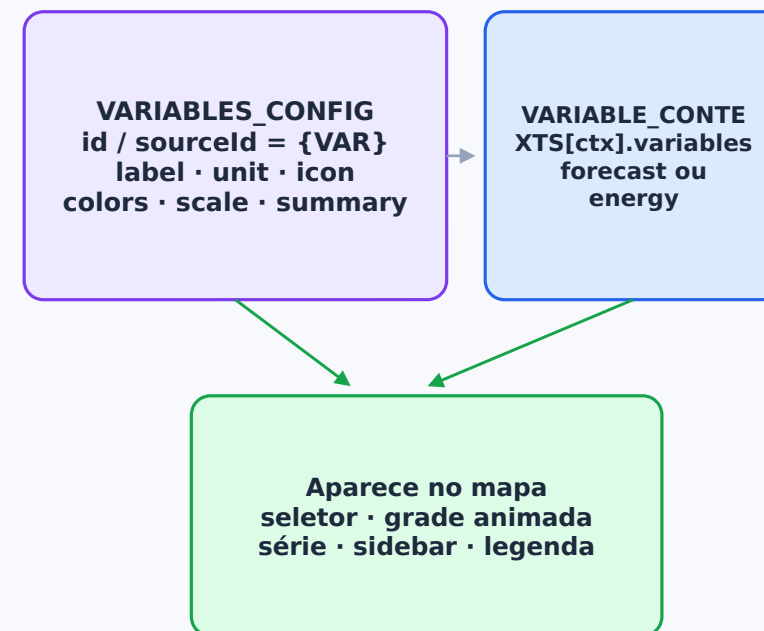
Adicionar uma variável nova aos mapas interativos

Do wrfout ao seletor do WebGIS - dois repositórios ligados por um contrato de ID

REPO 1 · micrometeorology - produz os dados



REPO 2 · site-labmim - mostra a variável



Contrato de ID
{VAR} =
VARIABLE_NETCDF_M
AP[nome]
ou nome.upper()

Resumo

Campo cru = DEFAULT_VARS + VARIABLES_CONFIG. Derivada = extractor + enum/ID + dispatch + DEFAULT_VARS + config do site.

Receita A: campo escalar cru (2 edições)

PIPELINE · cli/export_wrf_geojson.py

```

DEFAULT_VARS = [
    "temperature", "wind", "rain",
    # ... adicione o nome NetCDF:
    "SWDOWN", # radiação de onda curta
]

# O ramo genérico em jobs.py já cobre:
# ds.has_variable("SWDOWN") -> extract_scalar
# id de saída = "SWDOWN".upper()

$ labmim-wrf-geojson -v SWDOWN \
  -o site/JSON -g site/GeoJSON -D 1,4
# -> D01_SWDOWN_007.json
# -> D01_SWDOWN.series.bin

```

SITE · variables-config.js

```

globalRadiation: {
  id: "SWDOWN",
  sourceId: "SWDOWN", // == {VAR}
  label: "Radiação Global",
  unit: "W/m²",
  faIcon: "sun",
  colors: ["#fff", "#ffd700", /* ... */ "#7a0000"],
  scaleMin: 0,
  scaleMax: 1200,
  summary: "Onda curta incidente na superfície.",
  specificInfo: (v) => v == null
    ? unavailableInfo("Radiação")
    : { /* ... */ },
},

// Expor "globalRadiation" em
// VARIABLE_CONTEXTS.forecast.variables

```

1 · Registrar

DEFAULT_VARS

2 · Exportar

labmim-wrf-geojson -v
SWDOWN

3 · Configurar o mapa

VARIABLES_CONFIG + contexto

Receita B: variável derivada (uma edição por camada)

1 · EXTRACTOR · wrf/variables.py

```
def extract_relative_humidity(ds):
    q2 = ds.get_variable("Q2") # kg/kg
    t2 = ds.get_variable("T2") # K
    psfc = ds.get_variable("PSFC") # Pa
    rh = compute_relative_humidity(q2, t2, psfc)
    lo, hi = percentile_scale_bounds(rh)
    return rh, lo, hi
```

3 · DISPATCH · wrf/jobs.py

```
if variable_name == WRFVariable.RELATIVE_HUMIDITY:
    rh, vmin, vmax = variables.extract_relative_humidity(dataset)
    return ValueFrameSource(
        frame_for_step=lambda i: variables.materialize_2d(rh[i:i+1]),
        scale_min=vmin,
        scale_max=vmax,
    )
```

2 · TIPO + ID · common/types.py

```
class WRFVariable(StrEnum):
    RELATIVE_HUMIDITY = "relative_humidity"

VARIABLE_NETCDF_MAP = {
    WRFVariable.RELATIVE_HUMIDITY: "RH2",
}
```

4 · HABILITAR · export_wrf_geojson.py

```
DEFAULT_VARS = [ ..., "relative_humidity" ]

# Depois, no site:
sourceId: "RH2"
# exatamente o {VAR} do arquivo
```

Depois, no frontend

Crie a entrada em VARIABLES_CONFIG com sourceId: "RH2" e exponha a chave no contexto forecast ou energy.

Contrato e armadilhas ao expandir variáveis

ID

O contrato de ID

O {VAR} de {D}_{VAR}_{NNN}.json vem de VARIABLE_NETCDF_MAP[nome] ou nome.upper().

O sourceld no site precisa ser exatamente esse {VAR}.

Confira em JSON/manifest.json.

404

404 silencioso

sourceld errado faz o cliente pedir um JSON inexistente. O 404 entra em cache negativo e a variável fica muda sem erro visível.

Ícone novo: regenere o subset Font Awesome em scripts/subset-fontawesome.md; caso contrário, aparece uma caixa vazia.



Escala e paleta

colors[] é uma rampa clara -> escura. A cor por célula interpola entre scaleMin e scaleMax fixos no site.

Isso é decisão editorial e é diferente dos bounds por passo calculados no export com percentil 98.

Fontes

wrf/variables.py · wrf/jobs.py · common/types.py · cli/export_wrf_geojson.py · site/assets/js/variables-config.js

Referências rápidas

Comandos, documentação e caminhos para continuar a manutenção sem depender desta apresentação.

Comandos - site

```
npm run sites:list  
npm run build -- --site=<id>  
npm run build:all  
npm run build:check  
npm run lint:all
```

Pipeline de dados

```
labmim-wrf-geojson -v <var> \  
  -o site/JSON -g site/GeoJSON -D 1,4  
  
python -m  
micrometeorology.cli.run_wrf_pipeline
```

Documentação e código

```
site-labmim/Architecture.md  
src/sites/README.md  
scripts/site-builder/  
src/template/page-types.js  
  
micrometeorology/README.md  
docs/micrometeorology.md  
wrf/variables.py · wrf/jobs.py · common/types.py
```

github.com/Bruno-Mascarenhas/site-labmim

github.com/Bruno-Mascarenhas/micrometeorology

Fonte editável incluída: PPTX